

- IT Service Management (ITSM)
 - Introduction
 - Problem Statement
 - Data Cleaning — Observations
 - Exploratory Data Analysis (EDA) — Complete Observation Report
 - Feature Engineering — Observation Report
 - Model Building & Evaluation — Logistic Regression
 - Why Accuracy Was Not Used**
 - Model Building & Evaluation — Random Forest
 - Model Building & Evaluation — Decision Tree
 - Model Building & Evaluation — XGBoost
 - Final Model Evaluation — Confusion Matrix (XGBoost, Tuned)
 - Final Model Evaluation — ROC Curve (XGBoost, Tuned)
 - Final Model Evaluation — Feature Importance (XGBoost, Tuned)
 - Final Model Evaluation — Classification Report (XGBoost, Tuned)

IT Service Management (ITSM)

Introduction

ABC Tech is a mid-sized IT-enabled organization that has been operating for over a decade, handling an average of 22,000-25,000 IT incidents and tickets every month. The organization follows the ITIL (Information Technology Infrastructure Library) framework, managing its operations through structured incident management, problem management, change management, and configuration management processes.

While these ITIL practices have matured over the years, a recent internal audit revealed that further process-level improvements are unlikely to yield significant returns. At the same time, a customer satisfaction survey highlighted incident management as one of the weakest areas of service delivery, prompting the management to explore new avenues for improvement.

Recognizing the growing potential of Machine Learning (ML) in transforming ITSM (IT Service Management) operations, ABC Tech identified four key areas where predictive analytics could add value: forecasting high-priority incidents, projecting future ticket volumes, automating ticket tagging for faster routing, and predicting potential failures related to change requests (RFCs).

This project focuses on the first of these objectives — predicting high-priority IT incidents (Priority 1 and 2 tickets) using historical ticket data spanning 2012-2014, sourced from ABC Tech's internal ITSM database. The goal is to build a Machine Learning model capable of identifying high-priority incidents in advance, enabling the support team to take preventive action, reduce resolution delays, and ultimately improve overall service quality and customer satisfaction.

Problem Statement

Current Situation

ABC Tech, a mid-sized IT-enabled organization, handles approximately 22,000-25,000 IT incidents every month through its ITIL-based service management process.

Core Issue

- Despite following mature ITIL practices, recent customer feedback has rated the incident management process **poorly**.
- An internal audit confirmed that traditional process-improvement methods have reached a point of **diminishing returns**.
- High-priority incidents (Priority 1 and 2) are currently identified **only after they occur** — the process is entirely reactive.

Impact of the Problem

- Delayed response to critical incidents.
- Increased service disruption for end users.
- Inefficient allocation of support resources.
- Declining customer satisfaction scores.

Objective

To leverage historical incident data (2012-2014) and build a **Machine Learning classification model** that can predict whether an incoming IT ticket is likely to be a **high-priority incident**, using features such as: - Impact - Urgency - Configuration Item (CI) category - Reassignment history

Expected Outcome

By enabling early identification of high-priority tickets, ABC Tech aims to: - Reduce incident response time. - Minimize service disruption. - Improve overall efficiency and reliability of the IT incident management process.

Source

The dataset was extracted from a MySQL database (project_itsm) hosted on a remote server, accessed using read-only credentials provided as part of the project.

Method

- Connected to the database using **MySQL Workbench** (Standard TCP/IP connection).
- Queried the full incident table using `SELECT * FROM table_name;`
- Exported the query result set to a CSV file for local processing.

Challenges Faced

Issue	Cause	Resolution
Only 1,000 rows exported initially	MySQL Workbench's default "Limit Rows" setting	Increased Limit Rows Count to 100,000 in Preferences → SQL Editor → SQL Execution
ParserError: EOF inside string while loading CSV in pandas	Malformed quote characters in some text fields	Loaded data using <code>engine='python', quoting=3, on_bad_lines='skip'</code>

Final Extracted Dataset

- **Total records:** 46,606
- **Total columns:** 25
- **Verification:** Row count confirmed using `SELECT COUNT(*)` query,

matching the project documentation's stated dataset size (~46,000 records from 2012-2014).

- **File saved as:** data/raw/itsm.csv

Note

Data quality issues identified in columns such as Urgency, Impact, and Handle_Time_hrs (inconsistent formatting) were addressed separately in the **Data Cleaning** phase.

Data Cleaning — Observations

Initial Data Quality Check

On inspecting the extracted dataset (46,606 rows, 25 columns), several data quality issues were identified across multiple columns, requiring cleaning before proceeding to analysis.

Missing Values Summary

Column	Missing Count	% Missing	Action Taken
Reopen_Time	44,322	95.1%	Dropped column
No_of_Related_Incidents	45,384	97.4%	Dropped column
No_of_Related_Changes	46,046	98.8%	Dropped column
Related_Change	46,046	98.8%	Dropped column
CI_Cat, CI_Subcat	111 each	0.24%	Rows dropped (negligible impact)
Priority (target variable)	1,380	2.96%	Rows dropped — cannot impute the target label
No_of_Reassignments, Handle_Time_hrs	1 each	~0%	Rows dropped
Closure_Code	460	0.99%	Filled with "Unknown"
No_of_Related_Interactions	114	0.24%	Filled with median value
Resolved_Time	1,780	3.8%	Left as-is — likely represents tickets not yet resolved; excluded from modeling to avoid data leakage

Data Inconsistency: Urgency Column

The Urgency column was found to contain mixed data types — integers (4, 3, 5), string numbers ('4', '3'), and descriptive text labels ('5 - Very Low') — all representing the same underlying scale (1-5). This caused pandas to infer the column as an object dtype instead of a clean numeric type.

Fix: Extracted the leading numeric value from each entry using regex and converted the column to a consistent integer type. The Impact column was also checked for the same issue.

Data Corruption: Handle_Time_hrs Column

The Handle_Time_hrs column contained severely malformed values (e.g., "3,87,16,91,111" instead of an expected decimal value like 3871.69). The comma-based decimal notation (used in the source system) appears to have been affected by locale-based number formatting at the database/export level, resulting in Indian-style thousands-grouping being applied to what should have been a single decimal number. This corruption was found to originate from the source data itself, not from post-export handling.

Fix: Rather than attempting to repair the corrupted text values, Handle_Time_hrs was **recalculated from scratch** using the difference between Close_Time and Open_Time (converted to proper datetime format), expressed in hours. This approach was more reliable and avoided dependency on inconsistent source formatting.

Recalculated Handle_Time_hrs — Summary Statistics

Statistic	Value
Count	45,117
Mean	124.14 hrs
Median	18.80 hrs
Min	0 hrs
Max	15,312.32 hrs
Negative values	0

The distribution is **right-skewed** — most tickets are resolved within a day, while a small number of complex/stuck tickets remain open for extended periods (up to ~1.75 years), pulling the mean well above the median. No negative durations were found, confirming logical consistency between Open_Time and Close_Time.

Next Steps

- Convert Resolved_Time to proper datetime format for reference/analysis purposes (excluded from model features).
- Proceed to Exploratory Data Analysis (EDA) to visualize distributions, category-wise patterns, and relationships between Impact, Urgency, and Priority.

Exploratory Data Analysis (EDA) — Complete Observation Report

Overview

Following data cleaning, a comprehensive exploratory analysis was conducted on the ITSM dataset (45,117 records) to understand distributions, relationships, and patterns before proceeding to feature engineering and modeling. The analysis covered univariate distributions, outlier detection, categorical patterns, bivariate relationships, correlation analysis, and time-based trends.

1. Univariate Analysis — Numerical Columns

Histograms with KDE were plotted for Impact, Urgency, Priority, number_cnt, No_of_Reassignments, Handle_Time_hrs, and No_of_Related_Interactions.

Findings: - Impact, Urgency, and Priority show nearly identical distributions, with most tickets clustered around values **4 and 5**, and very few at **1 and 2** — indicating most incidents are lower severity. - number_cnt shows a roughly uniform distribution (0-1); its business meaning remains unclear and requires further verification. - No_of_Reassignments, Handle_Time_hrs, and No_of_Related_Interactions are all **strongly right-skewed**, with the majority of values low and a small number of extreme outliers pulling the tail far to the right.

2. Outlier Detection (Boxplots)

Boxplots confirmed significant outliers in the three right-skewed columns: - No_of_Reassignments: most tickets reassigned 0-2 times; outliers extend to 40+. - Handle_Time_hrs: most tickets resolved quickly; outliers extend beyond 14,000 hours. - No_of_Related_Interactions: mostly near 0; one extreme outlier reaches 370+.

These will be addressed during feature engineering (via capping, log-transformation, or reliance on tree-based models that are robust to skew).

3. Categorical Column Distributions

High-cardinality identifier-like columns (CI_Name, WBS, Incident_ID, KB_number, Related_Interaction) were excluded from distribution analysis, as they behave as unique identifiers rather than meaningful categories.

Column	Unique Values	Key Observation
Status	2	Dominated by "Closed" (~45,000+); "Work in progress" negligible "incident" dominates (~36,000), followed by "request for information" (~9,000); "complaint" negligible
Category	3	"application" most frequent (~32,000), followed by "subapplication"
CI_Cat	12	

CI_Subcat (top 15 shown)	62 total	(~7,500) “Server Based Application” and “Web Based Application” dominate, together accounting for a large share of all tickets
--------------------------	----------	---

Business Insight: Most IT incidents originate from the application/software layer rather than hardware.

Note: Alert_Status contains only **1 unique value** across the dataset (zero variance) — a candidate for removal before modeling.

4. Bivariate Analysis — Features vs Priority

Impact vs Priority & Urgency vs Priority: Both show a clear, consistent monotonic relationship with Priority, confirming the data follows the documented Priority Matrix logic. This validated that lower numeric values represent higher severity/priority (Priority 1 = highest priority), a scale convention retained as-is since it has no effect on model performance.

No_of_Reassignments, Handle_Time_hrs, No_of_Related_Interactions vs Priority: All three show heavy right-skew with large outliers, concentrated mainly in Priority 3-5. Notably, **Priority 1 and 2 tickets show tighter distributions with fewer outliers**, suggesting high-priority incidents are handled more promptly and consistently — a potentially useful predictive signal.

number_cnt vs Priority: Priority 1 shows a distinctly narrower range (0-0.65) compared to other priority levels (0-1) — business meaning still unclear.

5. Target Variable & Class Imbalance

The target variable `is_high_priority` was defined as: Priority $\in \{1, 2\} \rightarrow 1$, Priority $\in \{3, 4, 5\} \rightarrow 0$.

Class distribution:

Class	Count	Percentage
0 (Not High Priority)	44,425	98.46%
1 (High Priority)	692	1.53%

This represents **severe class imbalance**, meaning accuracy alone will not be a meaningful evaluation metric. Precision, Recall, and F1-score (particularly Recall for the minority class) will be prioritized during model evaluation, and imbalance-handling techniques (`class_weight='balanced'` and/or SMOTE) will be applied during model training.

6. Correlation Analysis

A correlation heatmap was generated for numeric/ordinal columns including the target variable.

Key findings: - Impact, Urgency, and Priority are extremely highly correlated with each other (0.98-0.99) and moderately correlated with `is_high_priority` (~0.39). However, this relationship is a direct artifact of the Priority Matrix formula (Priority is derived from Impact and Urgency, and `is_high_priority` is derived from Priority) — including these as model features would introduce **data leakage**. All three (Impact, Urgency, Priority) will therefore be **excluded from the feature set**. - `No_of_Reassignments`, `Handle_Time_hrs`, `No_of_Related_Interactions`, and `number_cnt` show very weak linear correlation with `is_high_priority` (all under 0.04). This does not necessarily indicate irrelevance — correlation only captures linear relationships, and tree-based models may still uncover non-linear patterns. Final feature relevance will be confirmed via feature importance after model training. - `No_of_Reassignments` and `Handle_Time_hrs` show a moderate correlation with each other (0.37), worth monitoring for multicollinearity.

7. Time-Based Trend Analysis

Monthly ticket volume was plotted using `Open_Time` across 2012-2014.

Finding: Overall ticket volume remains near-zero from early 2012 through mid-2013, followed by a sharp increase starting **September 2013** (jumping to 8,000+ tickets/month). High-priority tickets follow an identical pattern — **all 692 High Priority tickets fall exclusively within the September 2013-March 2014 window**, peaking in October 2013 (~175 tickets) before gradually declining.

Implication: This strongly suggests a data collection or system adoption gap in the earlier portion of the dataset (2012-mid 2013), rather than a genuine absence of incidents during that period. This is an important limitation, as the model's understanding of "High Priority" patterns is effectively derived from a concentrated ~7-month window rather than the full 3-year span, which may affect generalization.

Summary of Key Decisions Going Into Feature Engineering

- Impact, Urgency, and Priority will be **excluded from features** due to data leakage risk.
- `Alert_Status` will be **dropped** (zero variance).
- Identifier columns (`CI_Name`, `WBS`, `Incident_ID`, `KB_number`, `Related_Interaction`) will be **dropped**.
- Remaining categorical columns (`CI_Cat`, `CI_Subcat`, `Category`, `Status`, `Closure_Code`) will be **encoded**.
- `No_of_Reassignments`, `Handle_Time_hrs`, `No_of_Related_Interactions`, `number_cnt` will be **retained as candidate features**, subject to outlier treatment and feature importance validation.
- Class imbalance (98.46% vs 1.53%) will be addressed using `class_weight='balanced'` and/or SMOTE during model training.
- The concentrated time window of high-priority tickets (Sep 2013-Mar 2014) is noted as a data limitation affecting model generalization.

Feature Engineering — Observation Report

Overview

Building on the cleaned dataset (45,117 records) and EDA findings, feature engineering was performed to transform the data into a model-ready format. This involved removing irrelevant and leakage-prone columns, extracting new features from datetime fields, and encoding categorical variables.

1. Column Removal

The following columns were dropped prior to encoding:

Column(s)	Reason
CI_Name, WBS, Incident_ID, KB_number, Related_Interaction	High-cardinality identifier columns with no generalizable predictive pattern
Alert_Status	Zero variance (single unique value across all records)
Impact, Urgency, Priority	Excluded to prevent data leakage — Priority is directly derived from Impact and Urgency via the Priority Matrix, and the target variable (is_high_priority) is in turn derived from Priority. Including these would allow the model to reverse-engineer the target rather than learn genuine predictive patterns.
YearMonth	Helper column created only for EDA time-trend visualization; not needed as a model feature

2. Datetime Feature Extraction

Rather than dropping datetime information entirely, three new features were derived from Open_Time (the only timestamp available at the moment a new ticket is created, and therefore safe to use without leakage):

- open_month (1-12) — retained as a numeric feature to capture potential seasonal patterns.
- open_hour (0-23) — retained as a numeric feature to capture time-of-day patterns (e.g., business hours vs. after-hours).
- open_dayofweek (Monday-Sunday) — extracted as a categorical feature and one-hot encoded, since day-of-week is cyclical and has no meaningful numeric order.

Resolved_Time and Close_Time were **dropped without feature extraction**, since these timestamps only exist after a ticket is resolved/closed — using them (or any derived features) would constitute data leakage, as this information is unavailable at prediction time for a new incoming ticket.

3. Categorical Encoding

The following categorical columns were one-hot encoded using `pd.get_dummies()`: CI_Cat, CI_Subcat, Category, Status, Closure_Code, and open_dayofweek.

Special handling for CI_Subcat: This column originally contained 62 unique values, most of which were rare occurrences (long-tail distribution, as observed during EDA). To avoid excessive dimensionality from encoding, only the **top 15 most frequent sub-categories** (covering the vast majority of records — e.g., “Server Based Application” and “Web Based Application” alone account for over 33,000 of 45,117 tickets) were retained individually; all remaining sub-categories were grouped into a single “Other” category before encoding.

4. Final Feature Set

After column removal, feature extraction, and encoding, the dataset expanded from its original set of columns to **50 columns** (45,117 rows unchanged). All columns are now in numeric or boolean (True/False) format, with no remaining text/object columns, identifiers, or leakage-prone fields.

Final feature categories: - Numeric: number_cnt, No_of_Reassignments, Handle_Time_hrs, No_of_Related_Interactions, open_month, open_hour - One-hot encoded: CI_Cat_*, CI_Subcat_*, Category_*, Status_*, Closure_Code_*, open_dayofweek_* - Target: is_high_priority

Model Building & Evaluation — Logistic Regression

Overview

As the baseline model, Logistic Regression was trained using a pipeline combining StandardScaler (for feature scaling) and LogisticRegression. Given the severe class imbalance identified during EDA (98.46% vs. 1.53%), two versions of the model were trained and compared: one without imbalance handling, and one using `class_weight='balanced'`.

Since accuracy is a misleading metric under severe class imbalance, evaluation was based on **Precision, Recall, F1-Score, and ROC-AUC**, with particular focus on **Recall for the High Priority class**, as the business objective prioritizes catching high-priority tickets over minimizing false alarms.

Why Accuracy Was Not Used

Accuracy was intentionally excluded as an evaluation metric for this project. Given the severe class imbalance in the target variable (98.46% “Not High Priority” vs. 1.53% “High Priority”), a model that simply predicts “Not High Priority” for every single ticket would still achieve **98.46% accuracy** — despite being completely useless for the business objective of identifying high-priority incidents.

This makes accuracy a misleading metric in this context, as it does not reflect the model’s actual ability to detect the minority class (High Priority tickets), which is the entire purpose of this project. For this reason, evaluation was based on **Precision, Recall, F1-Score, and ROC-AUC** instead, which provide a more meaningful assessment of model performance on the imbalanced target.

Results Comparison

Metric	Without class_weight	With class_weight='balanced'
Precision	0.629	0.187
Recall	0.433	0.769
F1-Score	0.513	0.301
ROC-AUC	0.714	0.859

Observations

Without class_weight (default): The model achieved moderate precision (0.629) but low recall (0.433) — meaning it correctly identified only 43% of actual high-priority tickets, missing the majority of them. This behavior is expected, as the model was not adjusted for the underlying class imbalance and defaulted toward predicting the majority class.

With class_weight='balanced': Recall improved substantially to 0.769, meaning the model now correctly identifies ~77% of actual high-priority tickets — a significant improvement aligned with the business goal of minimizing missed critical incidents. ROC-AUC also improved (0.714 → 0.859), indicating better overall discriminating ability between classes.

However, this came at the cost of precision, which dropped sharply to 0.187 — indicating that roughly 81% of tickets flagged as “High Priority” by the model are false alarms. This is a classic **precision-recall trade-off**, commonly observed when addressing severe class imbalance.

Interpretation

The choice between the two versions depends on business priority: - **If missing a high-priority ticket is costlier than raising a false alarm** (consistent with the project’s stated goal of enabling preventive action before issues escalate), the `class_weight='balanced'` version is preferable, despite its lower precision. - **If minimizing unnecessary escalations and support-team workload is the priority**, the default (unweighted) version may be more suitable, at the cost of missing more high-priority tickets.

Both versions are reported here for transparency; the final model selection will be informed by comparing this trade-off against other algorithms (Random Forest, Decision Tree, XGBoost) before making a final recommendation.

Model Building & Evaluation — Random Forest

Overview

Following the Logistic Regression baseline, a **Random Forest Classifier** was trained and evaluated in two versions — without hyperparameter tuning, and with tuning via `RandomizedSearchCV` — to identify the best-performing configuration for predicting high-priority IT tickets.

Class imbalance (98.46% vs. 1.53%) was addressed using `class_weight='balanced'` in both versions. Hyperparameter tuning was optimized specifically for **Recall**, since — as established in the business case — **failing to catch a genuine high-priority ticket carries a higher business cost than raising an occasional false alarm.**

Results

Model	Precision	Recall	F1-Score	ROC-AUC
Random Forest (default parameters)	0.471	0.654	0.547	0.821
Random Forest (tuned for Recall)	0.348	0.712	0.468	0.845

Best tuned parameters: `n_estimators=100`, `max_depth=10`, `min_samples_split=10`, `min_samples_leaf=10`, `max_features=None`

Key Takeaways for Business Stakeholders

- **The tuned Random Forest model correctly identifies ~71% of all genuine high-priority tickets** (Recall = 0.712) — meaning roughly 7 out of every 10 critical incidents would be flagged proactively for preventive action, directly supporting the project's core objective.
 - **This comes with a trade-off:** when the model flags a ticket as "High Priority," it is correct only ~35% of the time (Precision = 0.348). In practical terms, **for every genuine high-priority ticket correctly caught, the support team will also receive roughly 2 false alarms** — tickets flagged as high priority that turn out not to be.
 - **The default (untuned) Random Forest offers a more balanced trade-off** — it catches fewer high-priority tickets (65%) but with noticeably fewer false alarms (47% precision), which may be more practical for day-to-day team workload if the support team cannot absorb a high false-alarm rate.
 - **Both Random Forest versions outperform the Logistic Regression baseline** in overall discriminating ability (ROC-AUC 0.82-0.85 vs. 0.71-0.86), confirming that the relationship between ticket attributes and high-priority status is **more complex than a simple linear pattern** — which Random Forest, as a non-linear model, captures more effectively.
 - **The choice between the two Random Forest versions is a business decision, not a technical one:** it depends on whether the organization prioritizes *catching more critical tickets* (tuned version) or *minimizing wasted effort on false alarms* (default version). This decision should be made in consultation with the IT support team's operational capacity.
-

Observations

The tuning process (RandomizedSearchCV, 50 combinations, 3-fold cross-validation) successfully improved both Recall and ROC-AUC compared to the default model, confirming that hyperparameter optimization adds measurable value. However, the drop in Precision illustrates the well-known **precision-recall trade-off** — improving one metric under severe class imbalance typically comes at the cost of the other, and no single “correct” answer exists independent of business priorities.

Model Building & Evaluation — Decision Tree

Overview

A **Decision Tree Classifier** was trained as a simpler, single-tree alternative to Random Forest. Three versions were evaluated: default parameters, tuning optimized for Recall, and a corrected tuning approach optimized for F1-Score. `class_weight='balanced'` was applied throughout to address the severe class imbalance (98.46% vs. 1.53%).

Results

Model	Precision	Recall	F1-Score	ROC-AUC
Decision Tree (default parameters)	0.164	0.740	0.268	0.841
Decision Tree (tuned — optimized for Recall)	0.086	0.779	0.155	0.825
Decision Tree (tuned — optimized for F1-Score)	0.448	0.625	0.522	0.806

Best parameters (F1-optimized): `max_depth=3`, `min_samples_leaf=15`, `min_samples_split=10`, `max_features=None`

Key Takeaways for Business Stakeholders

- **The final, corrected Decision Tree model correctly identifies ~63% of genuine high-priority tickets** (Recall = 0.625), and when it flags a ticket as high priority, it is **correct nearly 45% of the time** (Precision = 0.448) — meaning roughly 1 false alarm for every genuine catch. This is a substantially more usable and trustworthy result than the earlier tuning attempt.
- **An important lesson emerged during this stage of the project:** when hyperparameter tuning was initially optimized purely to maximize Recall, it produced a model that flagged far too many tickets as “high priority” — catching slightly more true positives (77.9% vs. 62.5%) but at the cost of an unusable false-alarm rate (only 8.6% precision, meaning roughly **11 false alarms**

for every correct catch). This version was discarded as impractical for real-world deployment, despite its higher Recall number.

- **This reinforces a critical principle for this project:** a single metric in isolation (such as Recall) can be misleading. Model selection must always weigh Recall against Precision and F1-Score together, guided by what the support team can realistically act on without becoming overwhelmed by false escalations.
- **The corrected Decision Tree now performs comparably to Random Forest** (F1-Score of 0.522 vs. 0.547 for default Random Forest), though it remains a single, simpler tree — offering an easier-to-interpret model at a small performance cost, which may be valuable if model transparency/explainability is a priority for the business.

Observations

This stage highlighted the importance of the **scoring metric used during hyperparameter tuning** — switching the optimization target from recall to f1 transformed an impractical, over-aggressive model into a well-balanced, usable one, without any change to the underlying algorithm or data. This finding will be applied to subsequent model tuning (XGBoost) to avoid the same pitfall.

Model Building & Evaluation — XGBoost

Overview

XGBoost, an advanced gradient-boosting algorithm widely used for tabular classification tasks, was trained and evaluated as the final algorithm in this project. Class imbalance was addressed using `scale_pos_weight=64` (calculated as the ratio of majority-to-minority class counts: 44,425 / 692). Both default and tuned versions were evaluated, with tuning optimized for **F1-Score** — a lesson carried forward from the Decision Tree stage, where optimizing purely for Recall was found to produce an impractical, false-alarm-heavy model.

Results

Model	Precision	Recall	F1-Score	ROC-AUC
XGBoost (default parameters)	0.416	0.596	0.490	0.792
XGBoost (tuned — F1-optimized)	0.491	0.649	0.559	0.819

Best tuned parameters: `n_estimators=400`, `max_depth=7`, `learning_rate=0.05`, `subsample=0.6`, `colsample_bytree=1.0`

Key Takeaways for Business Stakeholders

- **The tuned XGBoost model achieves the best overall balance of all models evaluated in this project**, correctly identifying ~65% of genuine high-priority tickets (Recall = 0.649), while being

correct roughly **49% of the time** when it flags a ticket as high priority (Precision = 0.491) — meaning close to 1 false alarm for every genuine catch, a substantially more manageable rate than several earlier model attempts.

- **Tuning provided a clear, consistent improvement here** (unlike the Decision Tree stage) — both Precision (+7.5 points) and Recall (+5.3 points) improved simultaneously, resulting in the highest F1-Score (0.559) achieved across all models and configurations tested in this project.
- **This model represents the recommended candidate for deployment**, offering the strongest trade-off between catching critical incidents early (supporting the project’s preventive-action goal) and keeping false escalations at a manageable level for the support team.

Final Model Comparison — All Algorithms

Model	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression (balanced)	0.187	0.769	0.301	0.859
Random Forest (default)	0.471	0.654	0.547	0.821
Random Forest (tuned)	0.348	0.712	0.468	0.845
Decision Tree (F1-tuned)	0.448	0.625	0.522	0.806
XGBoost (default)	0.416	0.596	0.490	0.792
XGBoost (tuned)	0.491	0.649	0.559	0.819

Overall Recommendation

XGBoost (tuned, F1-optimized) is selected as the final model for this project, based on achieving the highest F1-Score among all six models and configurations evaluated. It offers the most practical, deployable balance between catching high-priority tickets and minimizing false alarms — directly aligned with the project’s business objective of enabling preventive action without overwhelming the support team with unreliable escalations.

Notably, **Logistic Regression achieved the highest ROC-AUC (0.859) and highest Recall (0.769)** among all models, but its very low Precision (0.187) makes it impractical for real-world use on its own — this reinforces the project’s core methodology of evaluating multiple metrics together rather than relying on any single score to select a final model.

Final Model Evaluation — Confusion Matrix (XGBoost, Tuned)

Overview

The confusion matrix for the final selected model (XGBoost, F1-optimized) was generated on the held-out test set (13,536 tickets, stratified 70-30 split) to provide a detailed, ticket-level breakdown of model performance beyond the aggregate metrics reported earlier.

Confusion Matrix Results

	Predicted: Not High Priority	Predicted: High Priority
Actual: Not High Priority	13,188 (True Negatives)	140 (False Positives)
Actual: High Priority	73 (False Negatives)	135 (True Positives)

Key Takeaways for Business Stakeholders

- **Out of 208 genuine high-priority tickets in the test set, the model correctly identified 135 (65%)** — these are critical incidents that would have been proactively flagged for preventive action under this model, directly supporting the project’s core objective.
- **73 genuine high-priority tickets (35%) were missed** by the model — these tickets would have been treated as routine, receiving no early escalation. This is the primary risk associated with deploying this model: roughly 1 in 3 critical incidents may not be caught.
- **140 tickets were incorrectly flagged as high priority** when they were not — these represent the false-alarm cost of using this model. For context, this means the support team would need to review and dismiss 140 unnecessary escalations to catch the 135 genuine ones, a manageable but non-trivial additional workload.
- **The model correctly handled 13,188 out of 13,328 genuinely non-critical tickets (99%)** — confirming that the model does not disrupt the vast majority of routine ticket handling, and its impact is concentrated specifically around the high-priority decision boundary.
- **These figures translate directly to the Precision (49.1%) and Recall (64.9%) reported earlier** — the confusion matrix confirms these numbers are consistent and correctly calculated, providing a transparent, auditable basis for the model’s reported performance.

Business Interpretation

This breakdown makes the practical trade-off concrete: deploying this model would mean catching roughly two-thirds of critical incidents early, at the cost of occasionally investigating false alarms. Whether this trade-off is acceptable depends on the relative cost, to ABC Tech, of a missed critical incident versus the operational cost of reviewing a false escalation — a decision that should be made jointly with the IT support team based on their capacity to absorb additional review workload.

Final Model Evaluation — ROC Curve (XGBoost, Tuned)

Overview

The ROC (Receiver Operating Characteristic) curve was plotted using the model’s predicted probabilities, providing a threshold-independent view of the model’s ability to distinguish between High Priority and Not High Priority tickets.

Results

The model achieved an **AUC (Area Under Curve) of 0.926**, calculated using predicted probabilities rather than hard classification labels. This is notably higher than the 0.819 ROC-AUC reported earlier (which was calculated from binary predictions at the default 0.5 threshold), since probability-based AUC captures the model's full discriminating power across all possible decision thresholds, not just the single threshold currently in use.

Key Takeaway for Business Stakeholders

An AUC of 0.926 indicates strong underlying discriminating ability — the model is very effective at ranking tickets by their likelihood of being high priority. This is a strong signal that the model's core logic is sound; the earlier confusion matrix results (65% Recall, 49% Precision) reflect the specific threshold currently used to convert probabilities into a Yes/No decision, not a limitation of the model itself.

This opens an important possibility: adjusting the classification threshold (currently 0.5) could shift the balance between Precision and Recall to better match business needs — for example, lowering the threshold would catch more high-priority tickets (higher Recall) at the cost of more false alarms, while raising it would do the opposite. This is a tuning lever available for future refinement based on the support team's operational capacity.

Final Model Evaluation — Feature Importance (XGBoost, Tuned)

Overview

Feature importance scores were extracted from the final XGBoost model to identify which ticket attributes contribute most to predicting high-priority incidents. This provides interpretability alongside the model's performance metrics, and offers actionable business insight beyond a simple "black box" prediction.

Top Contributing Features

Rank	Feature	Importance
1	CI_Subcat_Banking Device	0.182
2	CI_Cat_networkcomponents	0.048
3	Category_incident	0.047
4	CI_Subcat_DataCenterEquipment	0.041
5	CI_Cat_storage	0.039
6	Closure_Code_User error	0.037
7	CI_Subcat_Windows Server	0.036
8	CI_Subcat_SAN	0.034
9	No_of_Related_Interactions	0.030
10	open_dayofweek_Saturday	0.029
11	open_dayofweek_Sunday	0.028
12	CI_Cat_subapplication	0.021
13	CI_Subcat_Laptop	0.021
14	Closure_Code_Referred	0.020

Key Takeaways for Business Stakeholders

- **“Banking Device” tickets are, by a wide margin, the single strongest predictor of high priority** (importance 0.182 — nearly 4x the next-highest feature). This suggests that incidents involving banking-related infrastructure are disproportionately likely to be classified as high priority, likely reflecting their direct impact on critical financial operations.
 - **Infrastructure-related categories dominate the top predictors** — network components, data center equipment, storage, Windows Servers, and SAN (Storage Area Network) all rank highly. This indicates that high-priority incidents are more strongly associated with core infrastructure failures than with end-user application issues, which is a useful signal for resource planning and proactive monitoring.
 - **Weekend tickets (Saturday and Sunday) show meaningful importance** — this suggests incidents reported on weekends carry a higher likelihood of being high priority, potentially because only critical issues get reported/escalated during off-hours when staffing is reduced, or because weekend incidents indicate more severe underlying problems.
 - **“Closure Code: User error” appearing as an important feature is a data consideration worth flagging** — since closure code is typically recorded only after a ticket is resolved, its presence as a predictive feature should be reviewed to confirm it does not introduce a subtle timing inconsistency in a live deployment scenario, even though it does not constitute the same direct leakage risk as Impact/Urgency/Priority.
 - **Business Recommendation:** ABC Tech may benefit from allocating additional preventive monitoring and support resources specifically toward banking devices, network components, and core data center infrastructure, as these categories are the leading drivers of high-priority incident classification in this model.
-

Observations

The feature importance results align well with intuitive business expectations — critical infrastructure and financial-system-adjacent categories rank highest, lending credibility to the model’s learned patterns beyond pure statistical performance. This reinforces that the model’s predictions are grounded in operationally meaningful signals rather than arbitrary correlations.

Final Model Evaluation — Classification Report (XGBoost, Tuned)

Overview

The full classification report provides a consolidated summary of Precision, Recall, and F1-Score for both classes, along with class-level support (number of actual instances), offering a complete picture of the final model’s performance on the test set.

Results

Class	Precision	Recall	F1-Score	Support
Not High Priority	0.99	0.99	0.99	13,328
High Priority	0.49	0.65	0.56	208
Accuracy			0.98	13,536
Macro Avg	0.74	0.82	0.78	13,536
Weighted Avg	0.99	0.98	0.99	13,536

Key Takeaways for Business Stakeholders

- **The model performs near-perfectly on the majority class** (Not High Priority: 99% precision, 99% recall) — this is expected given that this class makes up over 98% of all tickets, and confirms the model does not disrupt routine ticket handling.
- **The High Priority class — the entire purpose of this project — achieves 49% precision and 65% recall**, consistent with all previously reported results. This row is the one that matters most for business decision-making, as it reflects real performance on the rare, critical incidents this model was built to catch.
- **The overall 98% accuracy figure is included here only for completeness and should not be interpreted as a measure of success.** As established earlier in this project, a model that predicted “Not High Priority” for every single ticket would also achieve 98% accuracy while providing zero business value — this report confirms why accuracy was excluded from the model selection criteria throughout this project.
- **Macro Average (0.74 precision, 0.82 recall) treats both classes equally**, regardless of how many tickets fall into each — this gives a fairer sense of how the model handles the rare class specifically, without being diluted by the overwhelming volume of routine tickets. The gap between the Macro Average and the near-perfect Weighted Average (0.99) visually illustrates the class imbalance challenge this project was built around.
- **This report should be read alongside the confusion matrix and feature importance results already presented** — together, they confirm that the model’s performance on the “Not High Priority” class is not masking poor performance on the business-critical “High Priority” class, and that the reported 49%/65% figures are the true, honest measure of the model’s real-world usefulness.

Observations

This classification report consolidates and confirms all metrics reported throughout the model comparison stage, verified now on the true held-out test set for the final selected model. No new information is introduced here beyond what was previously reported per-metric; its value lies in presenting all figures together in the standard format typically expected in a machine learning evaluation summary.